JEV_RTL STUDY • KEV v1.0

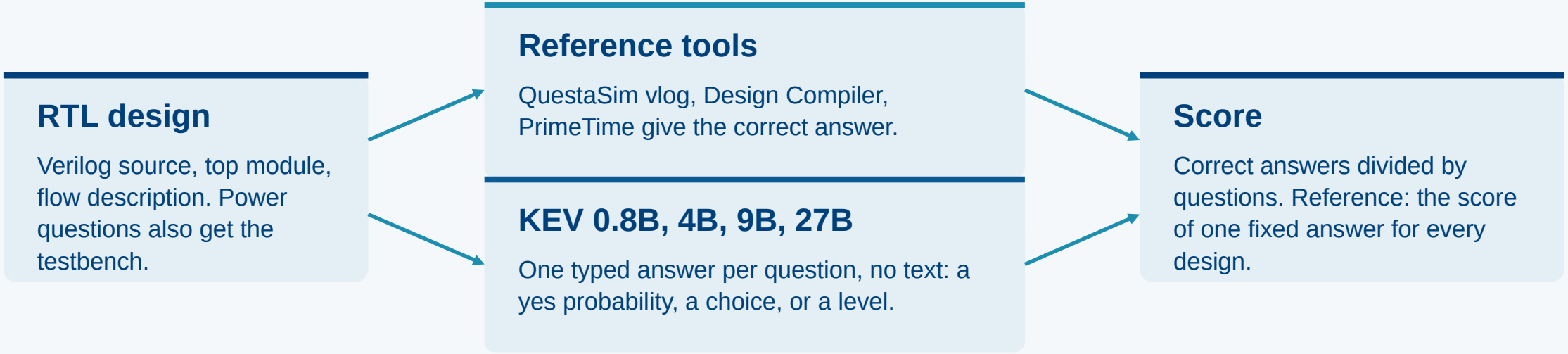
What can KEV tell us about an RTL design?

Eight questions, four model sizes, 123 test designs, without and with three solved examples

Runs of 15 October 2026 (no examples) and 16 October 2026 (three examples) • Deck of 21 October 2026

The test in one picture

The tools and KEV get the same RTL design. The score counts the KEV answers that agree with the tools.



Four KEV sizes

KEV v1.0 with 0.8B, 4B, 9B, and 27B parameters. Each size answers every question.

123 test designs

From 151 designs of five sources. The 28 development designs serve only as examples.

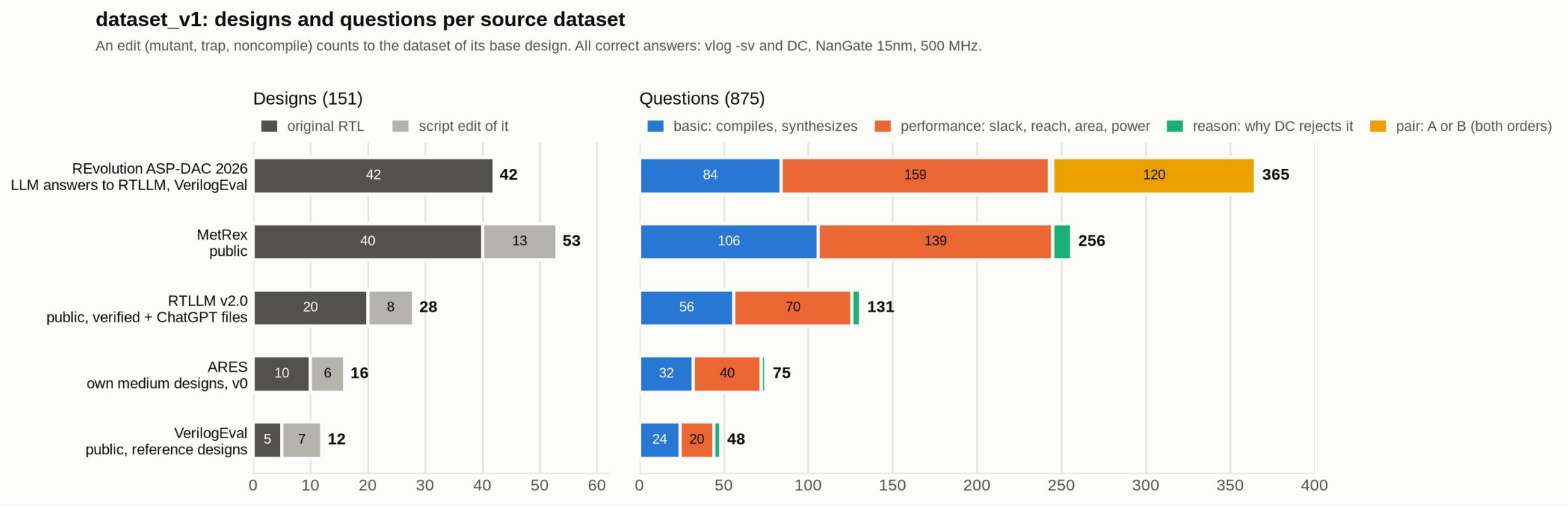
Two experiments

No examples (15 October). Three solved development examples before each question (16 October).

KEV never sees the tool logs. A tool run of each design gives the correct answer of every question about that design.

The designs: 151 designs from five sources

875 questions on 151 designs, both splits. An edit (mutant, trap, noncompile) counts for the source of its base design.



Test split: 123 designs, 725 questions. Development split: 28 designs, 150 questions, used only for the examples of experiment 2.

The eight questions

Each question has a typed answer. The correct answer of every question comes from a tool run of the same design.

Question	KEV answers with	Correct answer from	Test questions
Does it compile?	a yes probability	QuestaSim vlog -sv	123 designs: 114 yes, 9 no
Does it synthesize?	a yes probability	Design Compiler, compile_ultra	123 designs: 99 yes, 24 no
Why does synthesis fail?	one of ten reasons	the Design Compiler message	16 designs; the prompt says that it fails
Area level	one of five levels	Design Compiler, cell area	92 designs
Power level	one of five levels	PrimeTime, with the testbench	85 designs
Reachable clock frequency level	one of five levels	Design Compiler, worst slack	92 designs
Timing margin (slack) level	one of five levels	Design Compiler, worst slack	92 designs
Which of two designs has the smaller area, the lower power, the higher frequency?	A or B	the same tool runs	23, 22, 6 pairs, each asked in both orders

Compile and synthesize are two questions in one request; the power question of a design is in the same request as its area and timing questions.

The five levels of area, power, and timing

The model chooses a level from the source code and the flow description, not an exact value.

Question	Level 1	Level 2	Level 3	Level 4	Level 5
Area (gate equivalents)	below 100	100 to 1,000	1,000 to 10,000	10,000 to 100,000	100,000 or more
Power (microwatts)	below 10	10 to 100	100 to 1,000	1,000 to 10,000	10,000 or more
Timing margin, slack (ns)	negative	0 to 0.5	0.5 to 1.0	1.0 to 1.5	1.5 or more
Reachable frequency (GHz)	below 1	1 to 2	2 to 3	3 to 4	4 or more

Units

1 gate equivalent (GE) = the area of one NAND2 gate = 0.196608 µm² in NanGate 15 nm.
Power = dynamic + leakage power for the testbench stimulus.

Reachable frequency

Reachable frequency = 1 / (2 ns − worst slack), from the netlist synthesized at 500 MHz.
Example: 1.438 ns of slack gives 1 / 0.562 ns = 1.78 GHz.

One level of area or power is a factor of 10. No test design misses the 500 MHz target (slack level 1), reaches 4 GHz, or uses 10 mW or more.

Experiment 2: three solved examples per question

The same 725 test questions, one per request. The three examples are development designs with their tool result.

Question	The three examples before the design, with their tool result
Does it compile?	1 compiles, 2 fail; each failure with its first vlog error
Does it synthesize?	1 synthesizes, 2 fail; each failure with its first DC error
Why does synthesis fail?	multiple_drivers, illegal_code and event_list; each with its first DC error
Area level	71.5 GE (level 1) • 1,323.3 GE (level 3) • 519,474.0 GE (level 5)
Power level	12.48 µW (level 2) • 300.8 µW (level 3) • 26.2 mW (level 5)
Frequency level	612 MHz (level 1) • 2,137 MHz (level 3) • 3,572 MHz (level 4)
Slack level	+0.365 ns (level 2) • +0.962 ns (level 3) • +1.532 ns (level 5)
Which of two designs?	The three designs of the metric (area, power, or frequency), with the exact value only

All questions of one kind get the same three examples. No example shares a base design with a test design.

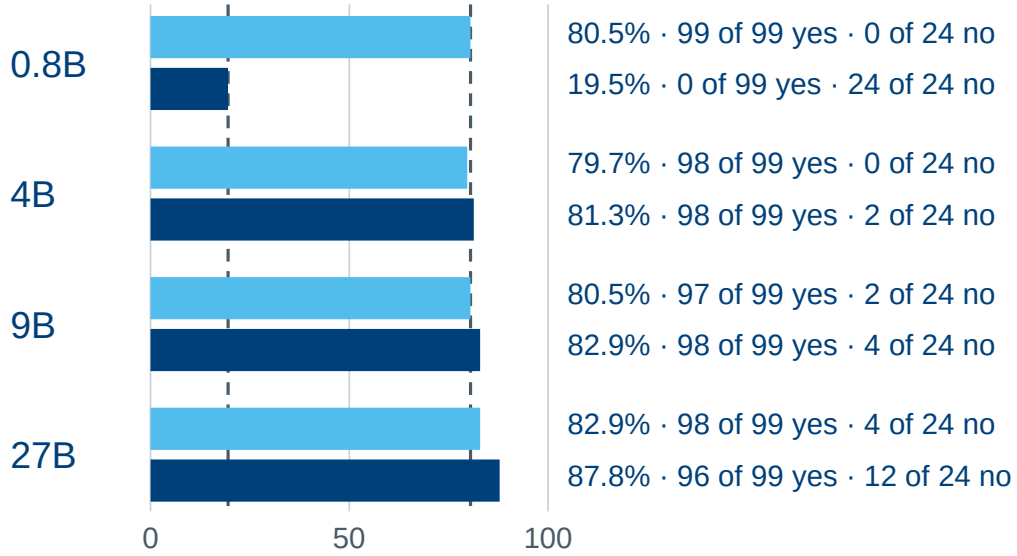
How to read the result charts

The same chart form on every result slide. The example: the synthesis question.

no examples three examples

Does it synthesize?

The tool says “yes” for 99 of the 123 designs and “no” for 24



Dashed lines: always “no” = 19.5% · always “yes” = 80.5%

Two bars per model

Light bar: the score without examples (experiment 1).
Dark bar: the score with three solved examples (experiment 2).

The label beside a bar

The score, then the correct answers on the designs where the tool says “yes”, and on the designs where it says “no”.

The dashed lines

The score of a model that always answers “no”, and of one that always answers “yes”. A bar at a dashed line shows no skill.

The score is the share of the test questions with a correct answer. Each question has the same weight.

The findings in one table

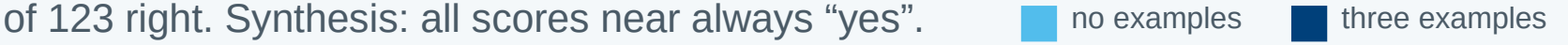
KEV-27B, the largest size. Each score: correct answers in % (count). The other sizes are on the result slides.

Question	No examples	Three examples	Always the same answer	Finding
Does it compile?	99.2% (122 of 123)	95.1% (117 of 123)	“yes”: 92.7%	Reliable without examples
Does it synthesize?	82.9% (102 of 123)	87.8% (108 of 123)	“yes”: 80.5%	Close to always “yes”
Why does synthesis fail? (told that it fails)	93.8% (15 of 16)	87.5% (14 of 16)	one reason: 31.3%	Good, when told that it fails
Area level	66.3% (61 of 92)	52.2% (48 of 92)	level 1: 35.9%	Usable; all within one level
Power level	10.6% (9 of 85)	42.4% (36 of 85)	level 2: 43.5%	Close to the constant answer
Frequency level	14.1% (13 of 92)	39.1% (36 of 92)	level 4: 48.9%	Below the constant answer
Slack level	32.6% (30 of 92)	65.2% (60 of 92)	level 5: 68.5%	Not above the constant answer
Smaller area of two designs	78.3% (36 of 46)	89.1% (41 of 46)	A: 50.0%	Good with examples
Lower power of two designs	70.5% (31 of 44)	79.5% (35 of 44)	A: 50.0%	Good with examples
Higher frequency of two designs	83.3% (10 of 12)	91.7% (11 of 12)	A: 50.0%	Good; only 6 pairs

KEV-27B is reliable on the compile check, and good on the failure reason and on the better of two designs. It is weak on the synthesis check and on the power and timing levels.

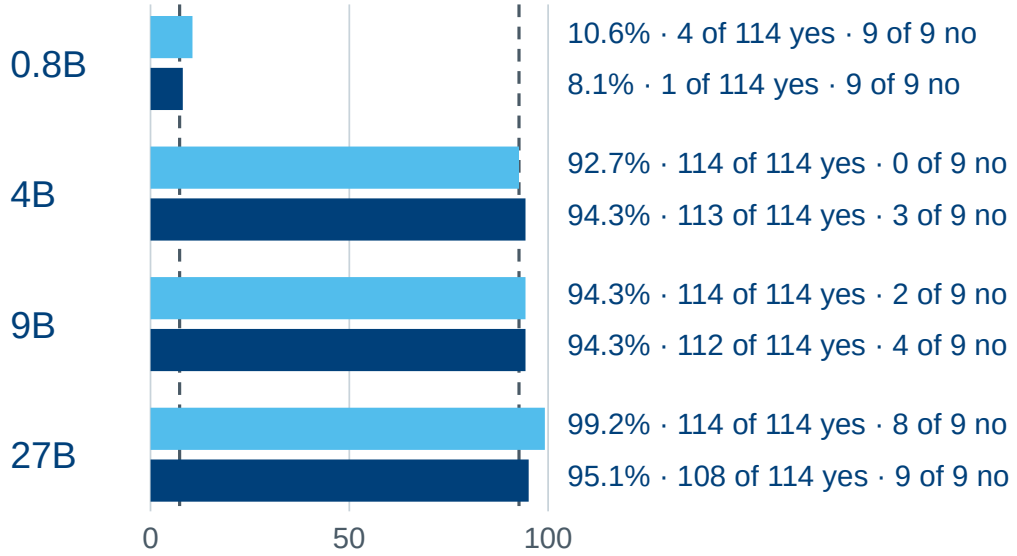
Yes/no checks: only the compile check is reliable

Compile: KEV-27B gets 122 of 123 right. Synthesis: all scores near always “yes”.



Does it compile?

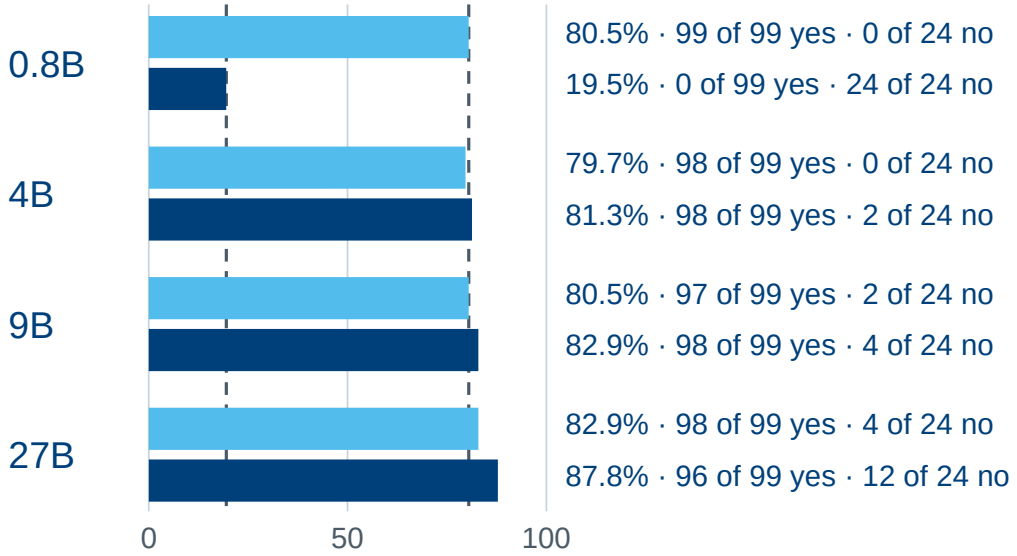
The tool says “yes” for 114 of the 123 designs and “no” for 9



Dashed lines: always “no” = 7.3% · always “yes” = 92.7%

Does it synthesize?

The tool says “yes” for 99 of the 123 designs and “no” for 24

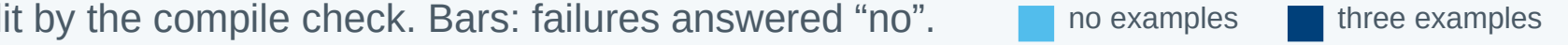


Dashed lines: always “no” = 19.5% · always “yes” = 80.5%

Synthesis: the best score, KEV-27B with examples, is 9 designs above always “yes”. KEV-0.8B with examples answers “no” to every design on synthesis.

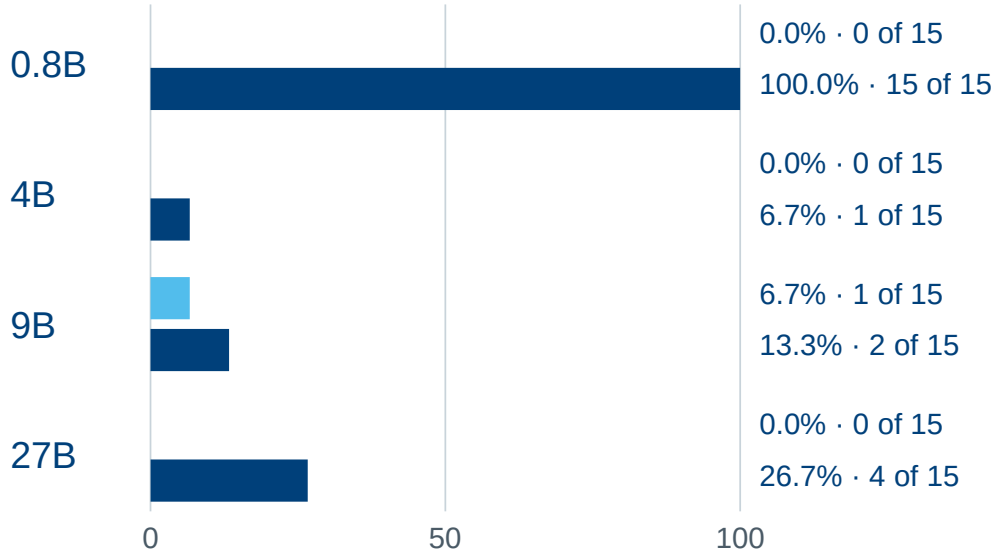
KEV misses most synthesis failures that compile

The 24 synthesis failures split by the compile check. Bars: failures answered “no”.



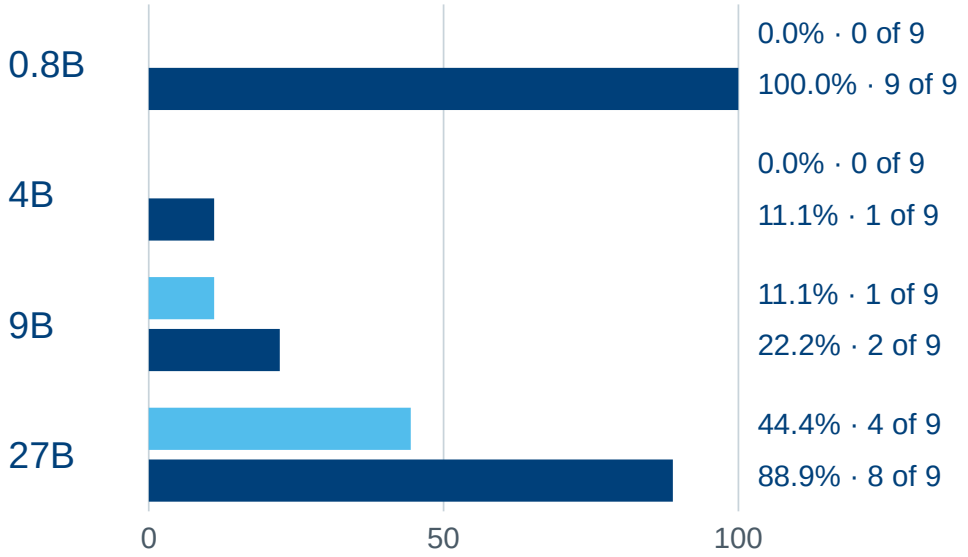
Failures whose source compiles

15 designs: only synthesis rejects them



Failures whose source does not compile

9 designs: vlog rejects them too



KEV-27B catches 4 of the 15 failures that compile with examples, 0 without; and 8 of the 9 that do not compile (4 without). KEV-0.8B’s full bars are a constant “no”.

The ten failure reasons and how often each occurs

The prompt says that synthesis failed. 25 reason questions: 16 in the test split, 9 in the development split (dev).

Reason	Text of the option, as the model reads it	Test	Dev
illegal_code	The code is not legal Verilog or SystemVerilog: a syntax error, an undeclared name, a non-constant range, or a procedural assignment to a net.	1	3
unsupported_construct	The code is legal for a simulator, but it uses a data type or construct that synthesis does not support: real, time, string, trireg, a class, a dynamic array, a DPI import, or a user-defined primitive.	1	0
timing_control	The code uses a timing control that synthesis does not support: wait, fork-join, or an event trigger (->).	3	1
simulation_statement	The code uses a statement or system function for simulation only: force-release, procedural assign-deassign, or $random.	5	1
event_list	An event list that synthesis does not support: both edges of one signal, edge and level events mixed, or different edges in one process.	0	1
async_reset	An asynchronous reset or set that synthesis cannot build: statements outside the reset if-else, a second edge signal without the reset template, or a reset to a non-constant value.	1	0
multiple_drivers	A signal has more than one driver, for example assignments in two always blocks.	1	2
mixed_assignments	One variable gets blocking and nonblocking assignments in the same always block.	2	0
unbounded_loop	A loop or a recursion that synthesis cannot unroll: a forever loop with no timing control, or a recursion with no constant limit.	0	1
instance_mismatch	A module instance does not match a module: a missing module, an unknown parameter, or an unknown port.	2	0

22 of the 25 failures are injected edits. 8 more test designs with an injected compile error get no reason question.

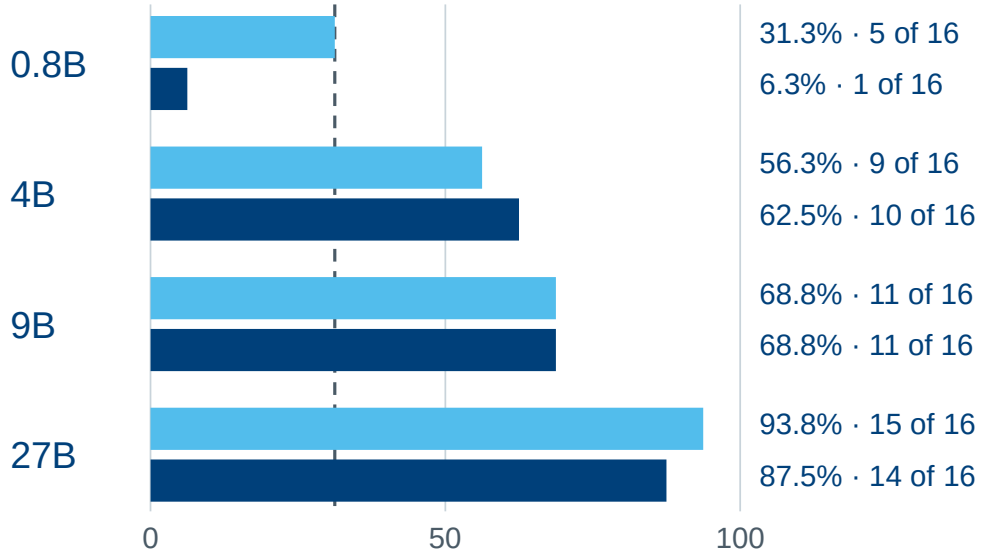
KEV-27B names a known failure but does not detect it

Left: the reason question. Right: synthesis and reason answers of the 16 designs.

no examples three examples

Why does synthesis fail?

16 designs · the prompt says that synthesis failed



Dashed line: always simulation_statement = 31.3% (5 of 16)

Model	Says that synthesis fails	Names the correct reason	Both
KEV-0.8B	0 → 16	5 → 1	0 → 1
KEV-4B	0 → 1	9 → 10	0 → 1
KEV-9B	1 → 2	11 → 11	1 → 2
KEV-27B	0 → 5	15 → 14	0 → 5

Each cell: designs of 16, no examples → three examples.
The synthesis answer comes from the yes/no request of the same design, where the prompt does not say that it fails.

KEV-27B names the reason for 15 of 16 known failures without examples, but it says “synthesis fails” for 0 of these 16 designs when the prompt does not tell it (5 with examples).

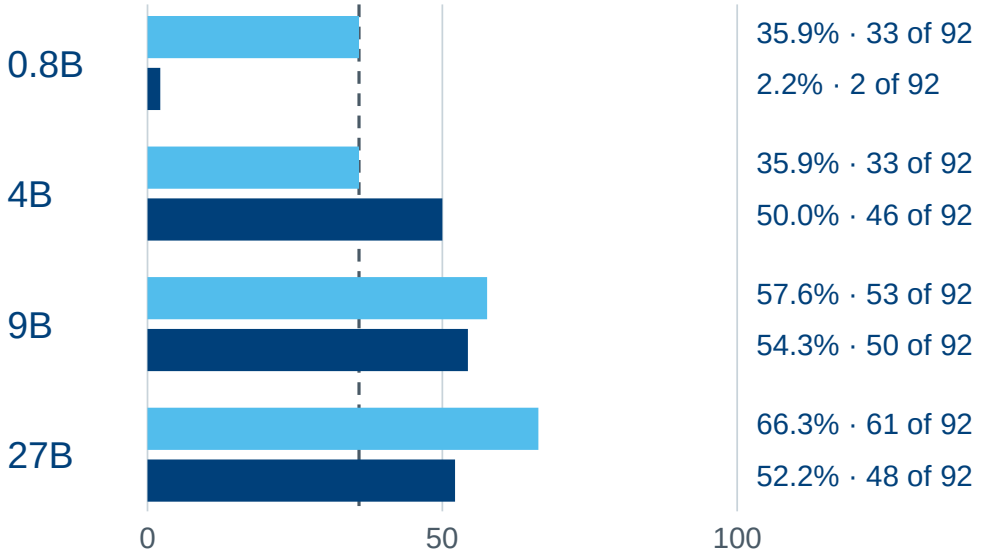
Area levels are usable; power levels are not

Bars: exact level. Dashed line: the most common level of the tool labels.



Area level

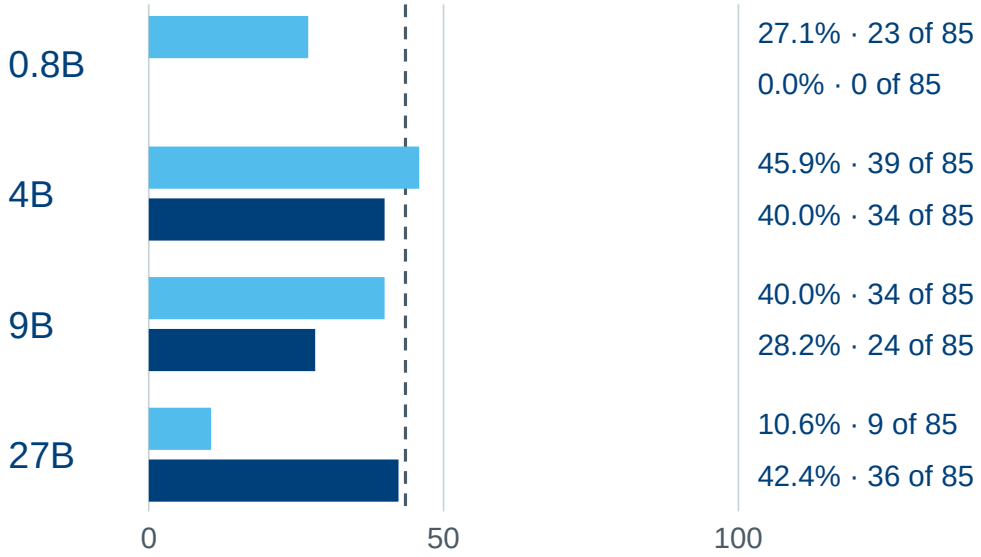
92 designs · all five levels occur



Dashed line: always level 1 = 35.9% (33 of 92)

Power level

85 designs · levels 1 to 4 occur



Dashed line: always level 2 = 43.5% (37 of 85)

Area: KEV-27B is exact for 61 of 92 designs and within one level for all 92; with examples the exact count drops to 48. Power: only KEV-4B without examples (39 of 85) is above always level 2 (37 of 85).

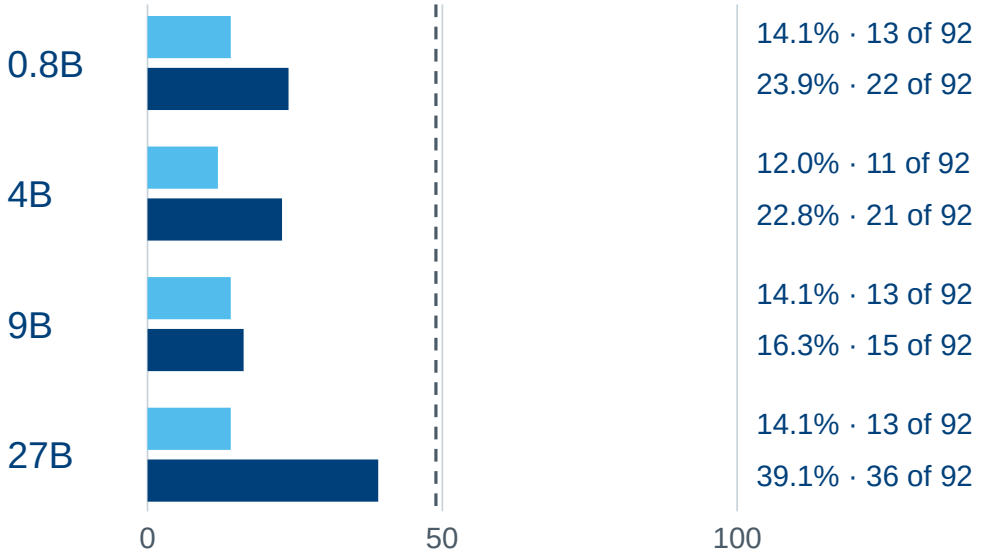
Timing levels: no model beats a constant answer

Bars: exact level. Dashed line: the most common tool level. No slack is negative.



Reachable clock frequency level

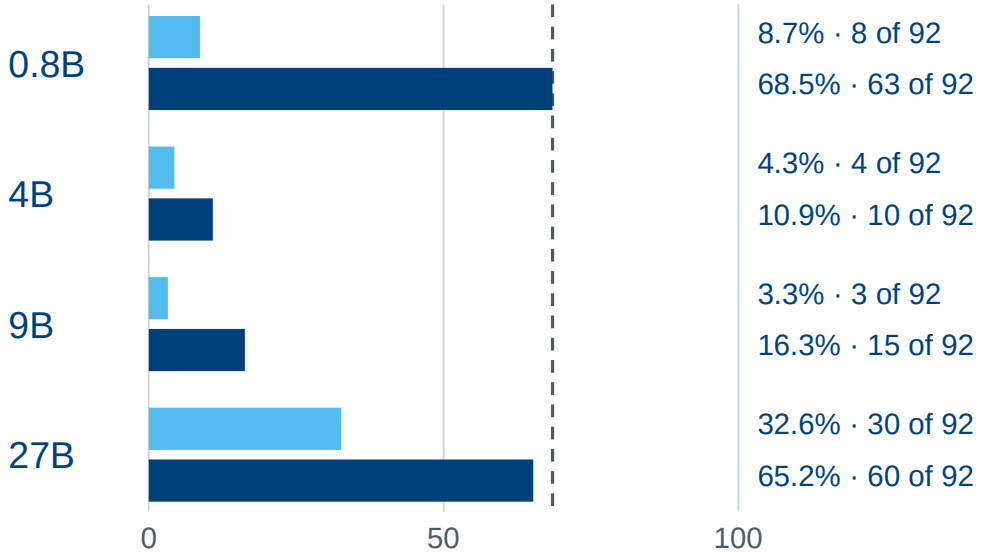
92 designs · levels 1 to 4 occur



Dashed line: always level 4 = 48.9% (45 of 92)

Timing margin (slack) level

92 designs · levels 2 to 5 occur



Dashed line: always level 5 = 68.5% (63 of 92)

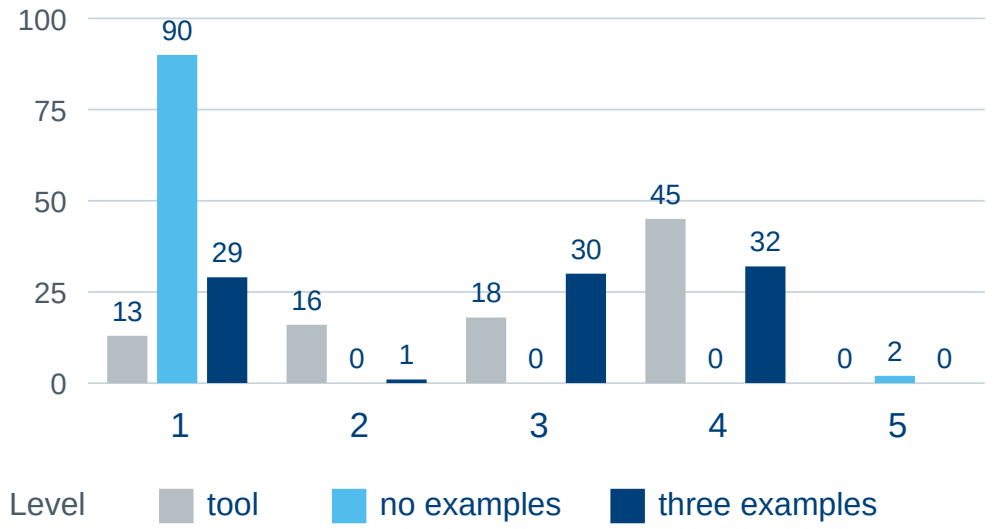
With examples KEV-27B reaches 36 of 92 (frequency) and 60 of 92 (slack); the constant answers score 45 and 63. KEV-0.8B’s 63 of 92 is level 5 for every design.

KEV-27B’s chosen levels do not match the tool’s levels

Designs at each level: the tool, then KEV-27B without and with examples. Equal counts are not correct answers.

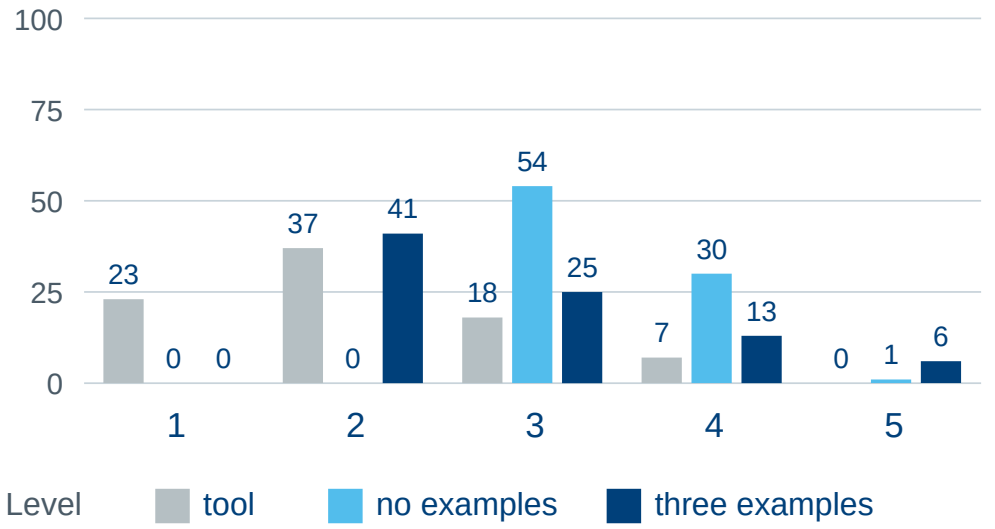
Reachable clock frequency level

92 designs · counts at each level



Power level

85 designs · counts at each level



Without examples KEV-27B puts 90 of 92 designs in frequency level 1 and no design in power level 1 or 2. With examples the counts come closer to the tool’s, but the exact scores stay below the constant answer.

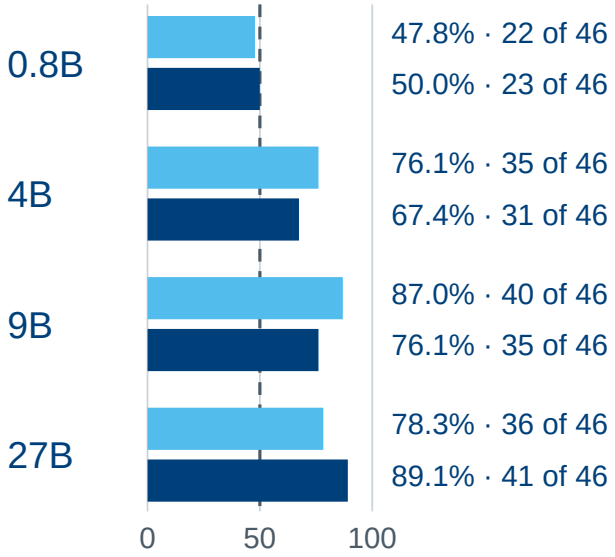
Pairs: KEV-27B with examples picks the better design

Two designs of one problem, A and B, asked in both orders. Always A scores 50%.



Smaller area

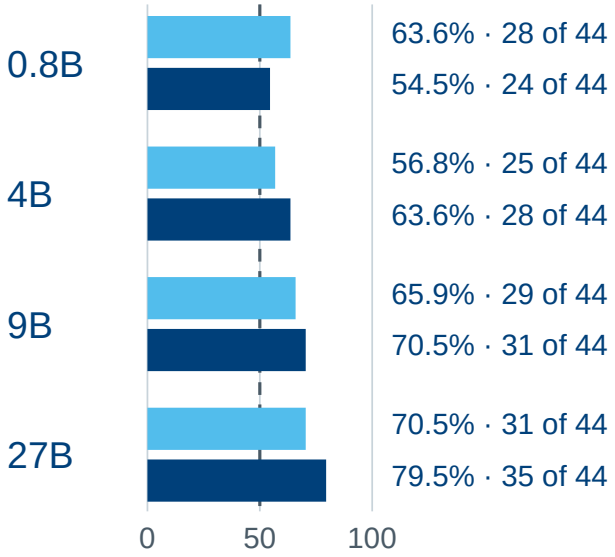
23 pairs · 46 questions



Dashed line: always A = 50%

Lower power

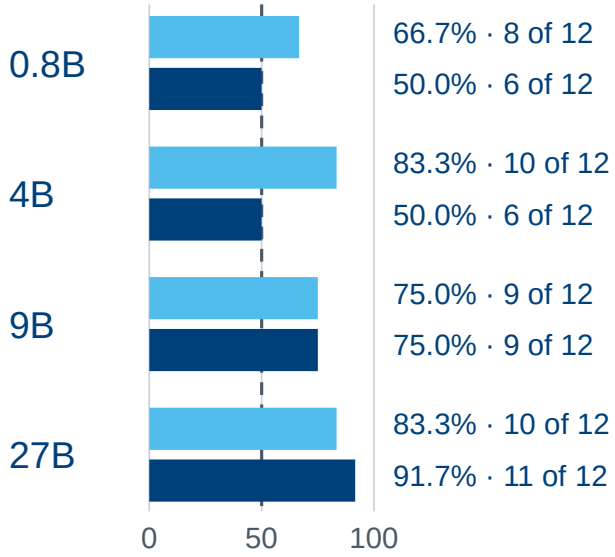
22 pairs · 44 questions



Dashed line: always A = 50%

Higher frequency

6 pairs · 12 questions



Dashed line: always A = 50%

With examples KEV-27B is right for 41 of 46 area, 35 of 44 power, and 11 of 12 frequency questions, and for 20 of 23 area pairs in both orders (17 without).

What the results support, and what to do next

Observations on dataset v1 and one fixed example set, not a guarantee for other designs or other examples.

Supported

- Compile check: KEV-27B, 122 of 123, no examples needed.
- Failure reason: KEV-27B, 15 of 16, when the prompt says that synthesis failed.
- Area level: KEV-27B, 61 of 92 exact, all 92 within one level.
- Better of two designs: KEV-27B with examples, 41 of 46 (area), 35 of 44 (power).

Not supported

- Synthesis check: at most 9 designs above always “yes”; KEV-27B misses 12 of 24 failures with examples, 20 without.
- Power, frequency, and slack levels: no score more than 2 designs above the constant answer.
- KEV-0.8B: one answer for every design in 6 of 10 questions with examples.

Next steps

- Dataset v2: more synthesis failures of code that compiles, designs that miss the timing target, equal yes and no counts.
- Other example sets, and more than three examples.
- Jev 1.13 and other models under the same protocol.
- Thresholds and example sets chosen on the development split only.

KEV does not replace the tools: the best synthesis check misses 12 of 24 failures. KEV can screen compile errors and rank two designs.

APPENDIX

Full results and reproduction

Every score of every size, the class counts, the probabilities, the level errors, the pair orders, the runs, and the files

Every score, without and with three examples

Each cell: correct answers in %, no examples → three examples. Bold: p < 0.05 (9 of 40 changes).

Question	n	0.8B	4B	9B	27B	Constant
Does it compile?	123	10.6% → 8.1%	92.7% → 94.3%	94.3% → 94.3%	99.2% → 95.1%	92.7%
Does it synthesize?	123	**80.5% → 19.5%**	79.7% → 81.3%	80.5% → 82.9%	82.9% → 87.8%	80.5%
Failure reason (told)	16	31.3% → 6.3%	56.3% → 62.5%	68.8% → 68.8%	93.8% → 87.5%	31.3%
Area: exact level	92	**35.9% → 2.2%**	35.9% → 50.0%	57.6% → 54.3%	**66.3% → 52.2%**	35.9%
Power: exact level	85	**27.1% → 0.0%**	45.9% → 40.0%	40.0% → 28.2%	**10.6% → 42.4%**	43.5%
Frequency: exact level	92	14.1% → 23.9%	12.0% → 22.8%	14.1% → 16.3%	**14.1% → 39.1%**	48.9%
Slack: exact level	92	**8.7% → 68.5%**	4.3% → 10.9%	**3.3% → 16.3%**	**32.6% → 65.2%**	68.5%
Pair: smaller area	46	47.8% → 50.0%	76.1% → 67.4%	87.0% → 76.1%	78.3% → 89.1%	50.0%
Pair: lower power	44	63.6% → 54.5%	56.8% → 63.6%	65.9% → 70.5%	70.5% → 79.5%	50.0%
Pair: higher frequency	12	66.7% → 50.0%	83.3% → 50.0%	75.0% → 75.0%	83.3% → 91.7%	50.0%

Constant: the score of the most common test label (always “yes”, one level, one reason, or one letter). The p value is the exact two-sided McNemar test on the paired questions; with 40 tests about 2 fall below 0.05 by chance.

Yes/no checks: correct answers per tool outcome

Each cell: designs of the group with a correct answer, no examples → three examples. “Yes” answers: of 123.

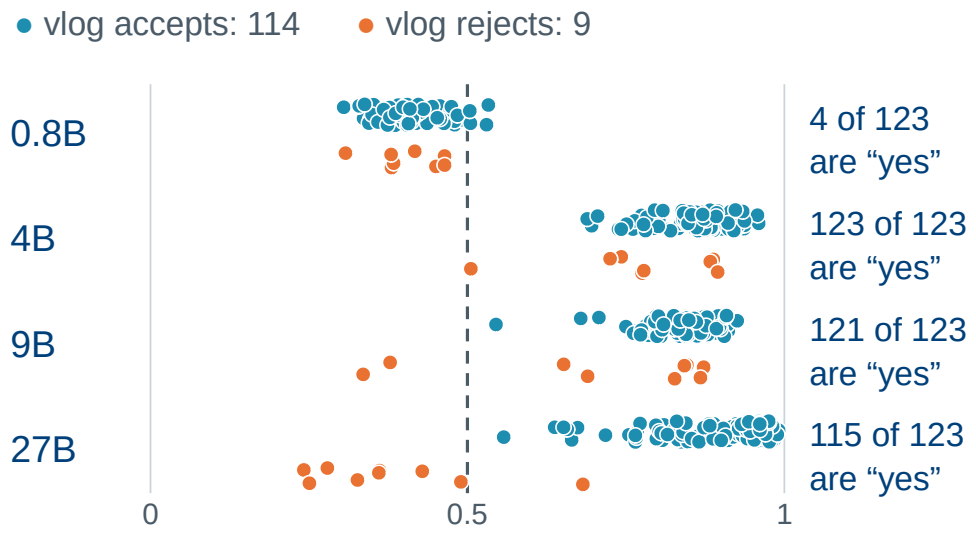
Group	Designs	0.8B	4B	9B	27B
Compile: the tool says “yes”	114	4 → 1	114 → 113	114 → 112	114 → 108
Compile: the tool says “no”	9	9 → 9	0 → 3	2 → 4	8 → 9
Compile: all designs	123	13 → 10	114 → 116	116 → 116	122 → 117
Compile: “yes” answers	123	4 → 1	123 → 119	121 → 117	115 → 108
Synthesis: the tool says “yes”	99	99 → 0	98 → 98	97 → 98	98 → 96
Synthesis: the tool says “no”	24	0 → 24	0 → 2	2 → 4	4 → 12
Synthesis: all designs	123	99 → 24	98 → 100	99 → 102	102 → 108
Synthesis: “yes” answers	123	123 → 0	122 → 120	119 → 118	118 → 108

“All designs” is the score of the result slides. The paired tests within each group are in the speaker notes.

Yes probability of every design, no examples

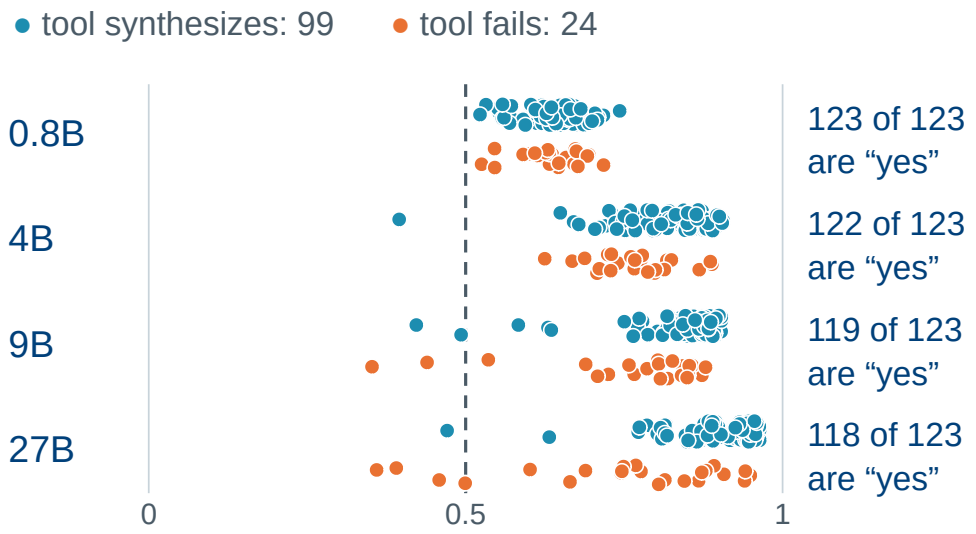
Each dot is one test design. 0.5 or more counts as “yes”. Teal: the tool says “yes”; orange: it says “no”.

Does it compile?



Yes probability of each design • dashed line: the 0.5 threshold

Does it synthesize?



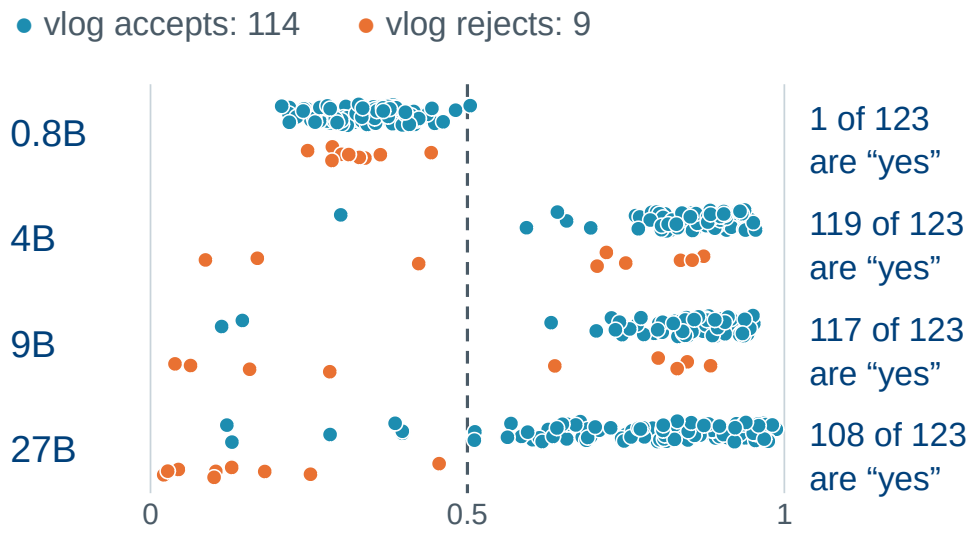
Yes probability of each design • dashed line: the 0.5 threshold

Only KEV-27B’s compile answers are right for most designs of both groups: 114 of 114 and 8 of 9.

Yes probability of every design, three examples

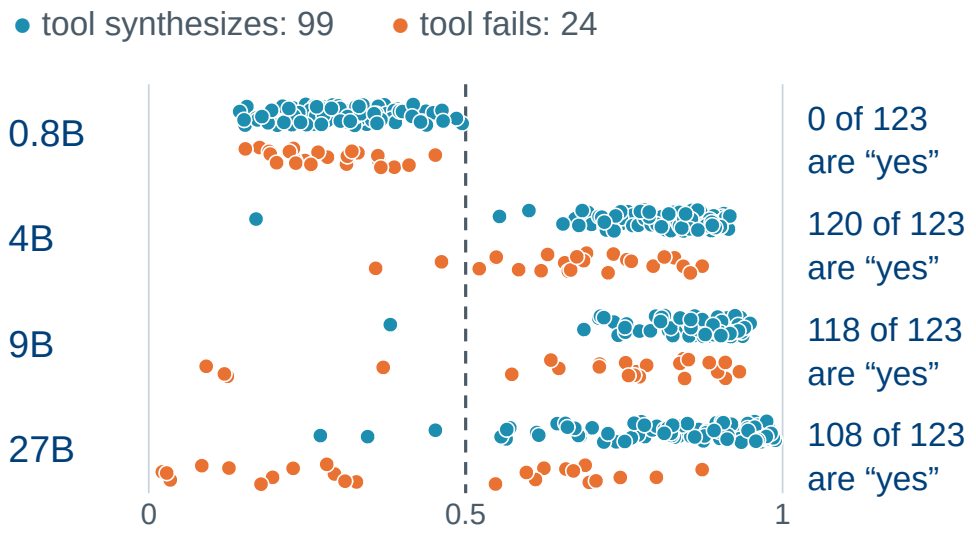
Each dot is one test design. 0.5 or more counts as “yes”. Teal: the tool says “yes”; orange: it says “no”.

Does it compile?



Yes probability of each design • dashed line: the 0.5 threshold

Does it synthesize?



Yes probability of each design • dashed line: the 0.5 threshold

Only KEV-27B’s compile answers are right for most designs of both groups: 108 of 114 and 9 of 9.

Failure reasons: correct answers per class

The 16 test designs by their true reason. Each cell: correct answers, no examples → three examples.

True reason	Designs	0.8B	4B	9B	27B
More than one driver	1	0 → 0	0 → 0	0 → 0	0 → 0
Illegal code	1	0 → 0	0 → 0	0 → 0	1 → 1
Unsupported construct	1	0 → 0	1 → 1	1 → 1	1 → 1
Unsupported timing control	3	1 → 1	3 → 3	3 → 3	3 → 3
Simulation-only statement	5	3 → 0	5 → 5	5 → 5	5 → 5
Asynchronous reset	1	0 → 0	0 → 0	0 → 0	1 → 0
Mixed assignment styles	2	0 → 0	0 → 0	0 → 0	2 → 2
Module-instance mismatch	2	1 → 0	0 → 1	2 → 2	2 → 2
Equal weight per class (balanced accuracy)		17.9% → 4.2%	37.5% → 43.8%	50.0% → 50.0%	87.5% → 75.0%

No test design has the reason event_list or unbounded_loop. The models chose event_list 6 times without examples and 16 times with examples (7, 3, 4, 2); it is one of the three example classes.

Ranking score and probability error, yes/no checks

A model can rank well and still decide badly at the 0.5 threshold. Cells: no examples → three examples.

Probability metric	0.8B	4B	9B	27B
Compile: ranking score (AUROC)	0.563 → 0.577	0.763 → 0.868	0.692 → 0.875	0.994 → 0.989
Compile: probability error (Brier)	0.334 → 0.420	0.064 → 0.056	0.066 → 0.057	0.030 → 0.068
Synthesis: ranking score (AUROC)	0.490 → 0.566	0.714 → 0.816	0.740 → 0.796	0.814 → 0.918
Synthesis: probability error (Brier)	0.187 → 0.416	0.148 → 0.130	0.144 → 0.123	0.128 → 0.088

Ranking score: higher is better

Chance = 0.5; perfect ordering = 1. The probability that a design that works gets a higher yes probability than a design that fails.

Probability error: lower is better

Mean squared error of the yes probability. Zero is perfect; a confident wrong answer costs more than an unsure one.

KEV-27B's synthesis ranking score rises from 0.814 to 0.918 with examples, while its failure detection at 0.5 rises from 4 to 12 of 24.

Within one level, and mean level error

Within one level: higher is better. Mean level error: lower is better. Each cell: no examples → three examples.

Question	Measure	0.8B	4B	9B	27B
Area	Within one level	66.3% → 12.0%	70.7% → 93.5%	90.2% → 92.4%	100.0% → 100.0%
Area	Mean level error	0.99 → 1.63	0.71 → 0.88	0.65 → 0.75	0.48 → 0.44
Power	Within one level	82.4% → 8.2%	91.8% → 88.2%	92.9% → 69.4%	64.7% → 87.1%
Power	Mean level error	0.97 → 1.61	0.75 → 0.99	0.90 → 1.06	1.07 → 1.04
Frequency	Within one level	31.5% → 62.0%	31.5% → 51.1%	33.7% → 51.1%	31.5% → 83.7%
Frequency	Mean level error	1.12 → 0.93	1.32 → 1.00	1.21 → 1.05	1.24 → 0.87
Slack	Within one level	30.4% → 85.9%	28.3% → 32.6%	12.0% → 76.1%	40.2% → 90.2%
Slack	Mean level error	1.51 → 0.76	1.65 → 1.44	1.74 → 1.47	1.62 → 0.92

Mean level error uses the probability-weighted expected level, not only the most likely level. The level ranges are decades, so one level is not a fixed physical error.

Pair questions: both orders, one order, neither

Counts per pair. Each pair has two answers, one per order. Each cell: no examples → three examples.

Pair outcome	Pairs	0.8B	4B	9B	27B
Area: both orders correct	23	5 → 0	13 → 10	19 → 12	17 → 20
Area: one order correct	23	12 → 23	9 → 11	2 → 11	2 → 1
Area: neither order correct	23	6 → 0	1 → 2	2 → 0	4 → 2
Power: both orders correct	22	6 → 3	3 → 8	9 → 11	14 → 17
Power: one order correct	22	16 → 18	19 → 12	11 → 9	3 → 1
Power: neither order correct	22	0 → 1	0 → 2	2 → 2	5 → 4
Frequency: both orders correct	6	3 → 0	4 → 0	4 → 4	5 → 5
Frequency: one order correct	6	2 → 6	2 → 6	1 → 1	0 → 1
Frequency: neither order correct	6	1 → 0	0 → 0	1 → 1	1 → 0

The two correct letters of a pair are opposite, so “one order correct” means the same letter in both orders. A letter swap can still be wrong in both orders.

Design groups in the test split

The retained designs by the group of the dataset plan. Pairs come from the REvolution candidates.

Design group	All retained	Test designs	Test synthesis failures
MetRex designs	40	31	0
Injected synthesis errors (mutants)	16	9	9
LLM-written candidates, natural result	20	17	0
Injected compile errors	8	8	8
Human-written reference designs	15	11	0
REvolution ASP-DAC 2026 candidates (pairs)	32	28	1
ARES v0 designs	10	9	0
Accepted edits and rejected twins (traps)	10	10	6

Not measured here: energy per operation, functional equivalence, post-route timing, measured chips, other model families, and the rename and comment controls of the earlier plan.

The Condor runs of the two experiments

Job times include the setup and differ in hardware: not a speed ranking. All requests answered, zero errors.

Experiment	Model	Job	GPU	Job time	Median model time per request
no examples	KEV-0.8B	4.0	RTX 3080 Ti	125 s	109 ms
no examples	KEV-4B	5.0	A100 80 GB	187 s	201 ms
no examples	KEV-9B	6.0	A100 80 GB	204 s	244 ms
no examples	KEV-27B	7.0	A100 80 GB	400 s	668 ms
three examples	KEV-0.8B	8.0	RTX 5060 Ti	276 s	179 ms
three examples	KEV-4B	9.0	RTX 3090	599 s	555 ms
three examples	KEV-9B	10.0	RTX PRO 5000	418 s	301 ms
three examples	KEV-27B	11.0	A100 80 GB	1074 s	1062 ms

No examples: 348 requests per model (both splits), maximum input 7,706 tokens. Three examples: 725 requests per model, maximum input 8,607 tokens; 3 requests exceed 8,192 tokens, the validated context of 0.8B, 4B, and 9B.

Files and checksums for reproduction

Every number is tied to the frozen question files and the saved answers. No model was rerun for this deck.

Model	Job, no examples	Completed	Job, three examples	Completed	Checkpoint revision (prefix)
KEV-0.8B	4.0	10-15 11:30	8.0	10-16 15:44	bf75a6a8848e
KEV-4B	5.0	10-15 14:31	9.0	10-16 15:49	6cfce5c2fa4b
KEV-9B	6.0	10-15 14:34	10.0	10-16 15:57	db029f08b290
KEV-27B	7.0	10-15 14:39	11.0	10-16 20:15	af0e6d551bdc

Inputs and evidence

Questions: data/questions/dataset_v1.jsonl and dataset_v1_fewshot3.jsonl (harness/fewshot.py)
Answers, scores, manifests, and comparisons: results/kev/dataset_v1_<model>_2026101[45]/ and results/kev/dataset_v1_fewshot3_<model>_20261016/
Verified tables and the build: presentations/data/verified_metrics.json, verified_metrics_fewshot3.json, compare_fewshot3.csv; presentations/src/analyse_kev.py, build_kev_findings.py